

Lab. Assignment 1 (04.08.2026)

1. Solve the Tic-Tac-Toe problem using an input matrix (traditional programming). Estimate the time complexity of the algorithm used.
2. Solve the water jug problem: You are given two jugs- a 4-li one and a 3-li one; a pump with unlimited water supply which you can use to fill the jugs, and the ground on which water may be poured. Neither jug has any measuring marks. How can you get exactly 2-li of water in the 4-li jug? Write the steps and code it.
3. Design and implement a Python program for a smart parking system. The parking area is represented as a two-dimensional grid containing an entrance, roads, available parking spaces, occupied parking spaces, and blocked areas. Use Breadth-First Search (BFS) to find the nearest reachable available parking space from the entrance. Display the selected parking position, the minimum number of movements required, and the route from the entrance to that space.

Problem Representation

The parking area is represented by a grid. A vehicle starts at the entrance and can move one cell at a time in four directions: up, down, left, or right. The vehicle may move only through road cells and available parking cells. Occupied spaces and blocked areas cannot be crossed.

Symbol Meaning

E Entrance or starting position

A Available parking space

O Occupied parking space

R Road through which the vehicle can move

X Blocked area or wall

Solution Using BFS

Breadth-First Search explores the parking grid level by level. It first checks all positions one movement away from the entrance, then positions two movements away, and so on. Since each movement has equal cost, the first available parking space found is one of the nearest reachable spaces.

1. Locate the entrance E and insert it into a queue.
2. Maintain a visited set so that each grid position is processed only once.
3. Remove the front position from the queue and examine its four neighbouring cells.
4. Ignore positions outside the grid, blocked cells X, occupied spaces O, and already visited cells.
5. For every valid neighbour, store its parent position and add it to the queue.
6. Stop when an available parking space A is reached.
7. Reconstruct the route by following parent links backwards from the selected space to the entrance, then reverse the route.

EXPECTED OUTPUT:

Example Grid

E	R	R	X	A
X	X	R	R	R
A	R	R	X	O
R	R	O	R	A

Nearest available parking space: (2, 0)

Number of movements required: 6

Route: (0, 0) -> (0, 1) -> (0, 2) -> (1, 2) -> (2, 2) -> (2, 1) -> (2, 0)

NOTE: The program should accept a parking grid of size at least 10×10 . The entrance, available spaces, occupied spaces, blocked locations, and roads should be provided as input. The program must use BFS to find the nearest reachable available parking space and display the route.